

CS 374: OPERATING SYSTEMS I

PRELIMINARIES

Mon/Wed 12:00 – 1:50 PM (LINC #2000)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

NOTICE

- Office hours
 - 22+ hours across 5 days a week
 - No office hours in the first week
 - Others: at Discord server

Office Hours										
Time	Mon	Tue	Wed	Thu	Fri					
10:00 AM	Jose	Jose	Jose							
10:30 AM										
11:00 AM				Sanghyun	Sanghyun					
11:30 AM										
12:00 PM										
12:30 PM										
1:00 PM		Bo	Bo	Bo						
1:30 PM										
2:00 PM					Jose					
2:30 PM										
3:00 PM	Jose									
3:30 PM										
4:00 PM										
4:30 PM										
5:00 PM	Jose									
5:30 PM										
6:00 PM										

RECAP

- Introduction to OS
 - What is an OS?
 - What are the functionalities of OS?

TOPICS OVERVIEW

- Part I: How OS runs programs?
 - Process and threads
 - Context switching
 - Virtual memory
 - Scheduling basics
- Part II: How OS loads/stores data?
 - Files
 - I/Os
 - Filesystem internals
- Part III: How OS support comm.?
 - Signals and PIPEs
 - Sockets
 - Networking I and II
- Part IV: How OS manages programs running on limited resources *safely*?
 - Synchronization I, II, and III
 - Deadlock
- Part V: How to make OS secure?
 - Rust
 - Rowhammer and side-channels

CONNECTING TO THE OS1 SERVER

CONNECT TO OS1 SERVER

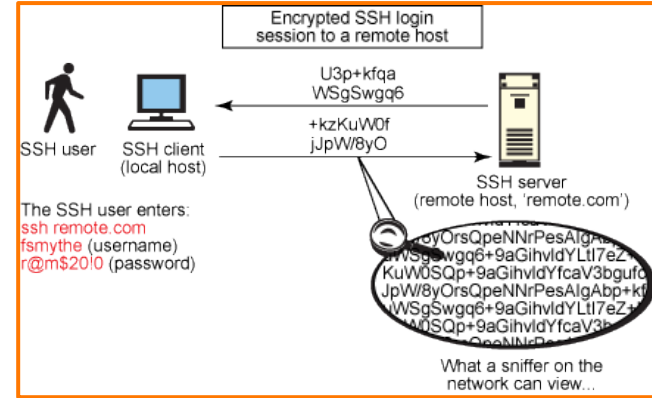
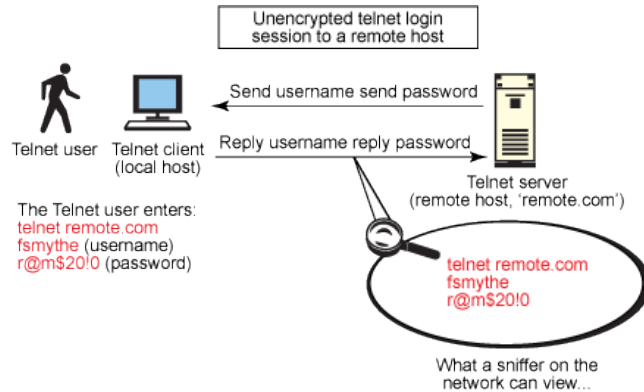
- Tools to connect: any SSH client
 - Terminal (Mac OSX), Terminal in VS Code (Mac OS / Windows)
- How to access the OS 1 server?
 - On-campus: `ssh <ONID>@os1.engr.oregonstate.edu`
 - Off-campus:
 - First, ssh to those: `ssh <ONID>@access/flip.engr.oregonstate.edu`
 - Second, ssh to the OS 1 server: `ssh <ONID>@os1.engr.oregonstate.edu`
 - Or you need the VPN (Cisco client)
 - Note: do *not* run any program on the access/flip servers

CONNECT TO OS1 SERVER – CONT'D

- SSH without password
 - Authentication using an SSH key
 - **Pro:** don't need to type password in every SSH log-in
- How to?
 - Generate a *private* and *public* key pair on your PC/laptop
 - Command: `ssh-keygen -t ed25519 -C "<ONID>@oregonstate.edu"`
 - Output: you will have <keyname> and <keyname>.pub under a specified folder
 - Copy the public key to the OS 1 server
 - Open <keyname>.pub and copy the content
 - Paste it into *authorized_keys* file in OS 1 server's <your home>/*.ssh* folder
 - Update the permission of *authorized_keys* file to 700: `chmod 0700 authorized_key`
 - Try SSH command again
 - `ssh <ONID>@access/flip.engr.oregonstate.edu` (It won't ask the password again)

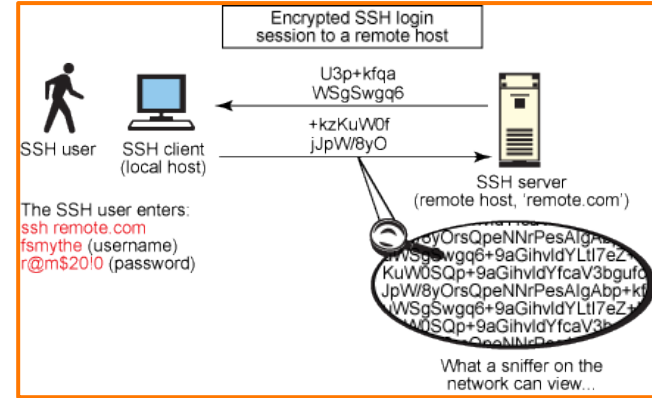
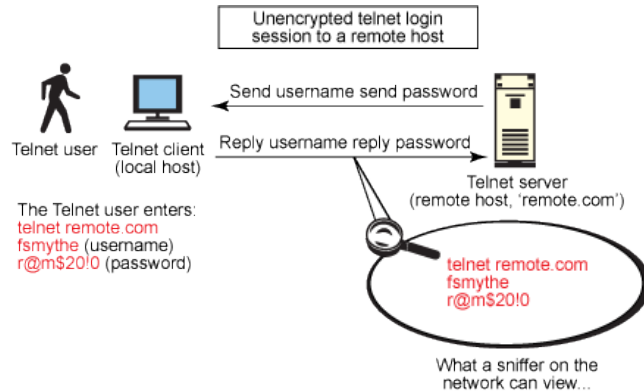
CONNECT TO OS1 SERVER – CONT'D

- How does it work and why do we do?
 - Password login is not secure against man-in-the-middle attackers
 - Potential solutions:
 - Encrypt login information
 - Encrypt all the communications (like German's Enigma)



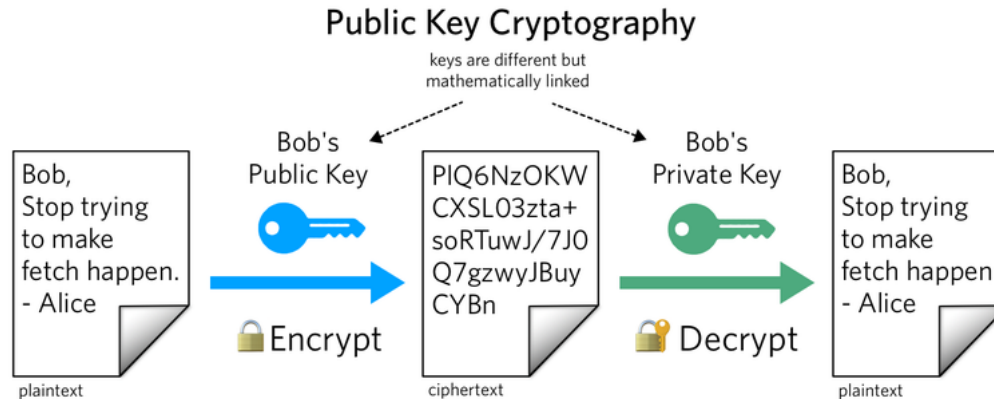
CONNECT TO OS1 SERVER – CONT'D

- How does it work and why do we do?
 - Password login is not secure against man-in-the-middle attackers
 - Potential solutions:
 - ~~Encrypt login information~~
 - Encrypt all the communications (like German's Enigma)
 - Encrypt, but **not with a shared mechanism (not with a shared key)**



CONNECT TO OS1 SERVER – CONT'D

- How does it work and why do we do?
 - Password login is not secure against man-in-the-middle attackers
 - Potential solutions:
 - **Asymmetric (public-key-based) encryption**
 - An authentication protocol with the asymmetric encryption



SYMMETRIC KEY CRYPTOGRAPHY

- Problems

- How can we securely share the key between two parties?
- How can we manage communications from/to multiple parties (100+)?



SYMMETRIC KEY CRYPTOGRAPHY

- Problems

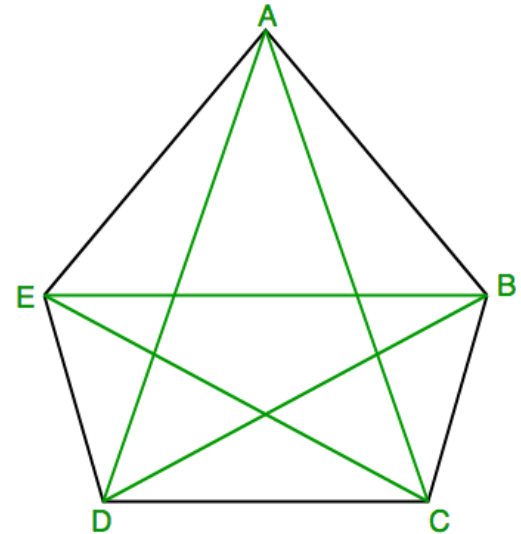
- How can we securely share the key between two parties?
- How can we manage communications from/to multiple parties (100+)?

- Solutions

- What if I have two keys?
 - Key A that only can encrypt a message (but can't decrypt)
 - Key B that can encrypt and decrypt a message
- How can I leverage the two keys?
 - Share Key A to others
 - Do not share; keep Key B private

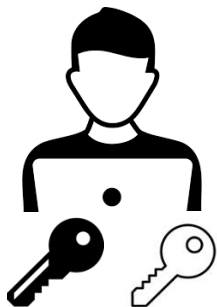
KEY EXCHANGE IN SYMMETRIC KEY CRYPTOGRAPHY

- Key exchange complexity
 - A key per each pair of people
 - $nC_2 = N(N - 1) / 2$
 - $O(N^2)$



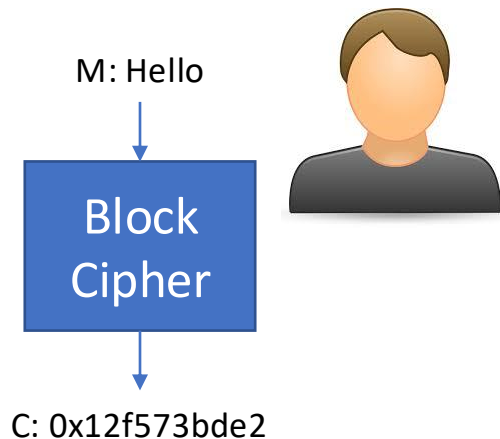
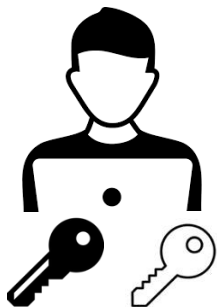
PUBLIC KEY CRYPTOGRAPHY

- The key idea
 - Asymmetric key cryptography
 - Use two different keys for encryption and decryption
 - Public key: share to others, only can encrypt a message
 - Private key: do not share, can encrypt and decrypt
 - What is possible?



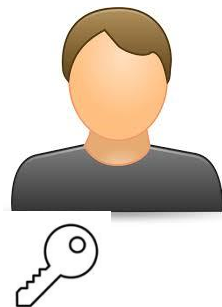
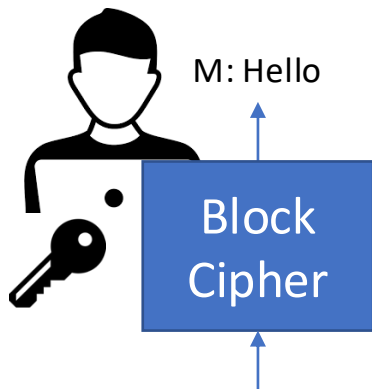
PUBLIC KEY CRYPTOGRAPHY

- The key idea
 - Asymmetric key cryptography
 - Use two different keys for encryption and decryption
 - Public key: share to others, only can encrypt a message
 - Private key: do not share, can encrypt and decrypt
 - What is possible?



PUBLIC KEY CRYPTOGRAPHY

- The key idea
 - Asymmetric key cryptography
 - Use two different keys for encryption and decryption
 - Public key: share to others, only can encrypt a message
 - Private key: do not share, can encrypt and decrypt
 - What is possible?

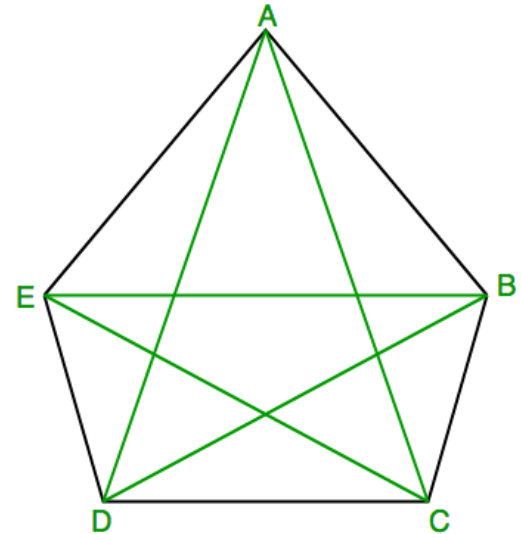


PUBLIC KEY CRYPTOGRAPHY

- The key idea
 - Asymmetric key cryptography
 - Use two different keys for encryption and decryption
 - Public key: share to others, only can encrypt a message
 - Private key: do not share, can encrypt and decrypt
 - What is possible?
 - No one can decrypt a ciphertext unless they have the private key
 - We do not need to share the private key to anyone else
 - We share public key that can only encrypt the message

PUBLIC KEY CRYPTOGRAPHY AND KEY EXCHANGE

- Suppose we have five people (A, B, C, D, E)
 - How many keys do we need to make them communicate securely?
 - How can we make everybody be able to talk to anyone?
- In block cipher
 - Need 1 key for two of them (A and B) can talk securely
 - How many keys do we need for all?
 - A-B, A-C, A-D, A-E
 - B-C, B-D, B-E
 - C-D, C-E
 - D-E
 - 10 keys (${}_5C_2 = 10$)



CONNECT TO OS1 SERVER – CONT'D

- How does it work and why do we do?
 - Password login is not secure against man-in-the-middle attackers
 - Potential solutions:
 - Asymmetric (public-key-based) encryption
 - **An authentication protocol with the asymmetric encryption**
 - 1) You put **your public key** to the server (manually)
 - 2) You ask the connection
 - 3) The server encrypts **a challenge** with **your public key** and send it to you
 - 4) You decrypts the package with **your private key**
 - 6) You solve **the challenge** and encrypt the answer with **your private key**
 - 7) The server decrypts the package with **your public key** and verifies **the answer**
 - 8) Both establish the *safe* connection and communicate with the encryption

LINUX COMMAND LINE BASICS

SHELL

- What is shell:
 - **Formal:** A program which exposes OS's services to users or to other programs
 - **Informal:** That you will see after the SSH log-in
- What are the types?
 - ***Bourne shell (bash)**, Korn shell (ksh), Z shell (zsh), C shell (csh)...
- What are the (basic) features?
 - Print a message(s)
 - Launch a program
 - Create, rename, or remove files and directories
 - See what programs running on OS



BASIC BASH SHELL COMMANDS

- Basic commands
 - Print a message(s): `echo`
 - Launch a program: `./<program name> <arguments>`
 - Create, rename, or remove files (and directories)
 - Create a dir: `mkdir (-p) <directory>`
 - Create a file: `touch <filename>`
 - Move a dir/file: `mv <file/directory> <destination>`
 - Copy a dir/file: `cp -rf <file/directory>`
 - Remove files/dirs: `rm (-r <empty directory> / -f <file> / -rf <all files and subdirectories>)`
 - Others:
 - List files or directories: `ls (-al / -lh / -t) <file or directories>`
 - Go to a certain directory: `cd <directory path>`
 - Print out a file content: `cat <filename>`
 - Print out a (log) file content being updated: `tail (-f) <filename>`

BASIC BASH SHELL COMMANDS – CONT'D

- Basic commands
 - Others
 - See what programs are running on OS: `ps (-ef)`
 - See who runs what programs on OS: `ps -ef | grep <username>`
 - See the OS version and distribution: `uname (-r / -a)`
 - See the CPU/mem.: `cat /proc/cpuinfo` (or `cat /proc/meminfo`)
 - See the directories where you are: `pwd` (absolute path)
 - ...

BASIC BASH SHELL COMMANDS – CONT'D

- Data wrangling
 - You can run multiple commands at once: `<command 1>; <command 2>; <command 3>...`
 - You can combine multiple commands at once:
 - Sequential executions: `<command 1> && <command 2>`
 - Store execution results of a command to a file: `<command 1> > <output file>`
 - Run a program background: `<command 1> &`
 - Example) run in background and store the results to a file: `<command 1> > <output file> &`
 - Example) see the output file in real-time: `tail -f <output file>`
 - More commands (with previous commands)
 - Search for files or directories: `find <directory> -name <token like *sanghyun*>`
 - Search for a string in files or directories: `grep -nr <token like sanghyun> <directory>`

Tips: Be Creative with Your Combinations!

BASIC BASH SHELL SETUP

- Customization
 - Use the configuration file: `~/.bashrc` or `~/.profile` (`~/` indicates your home dir.)
 - Add commands you want to run when you log-in: `echo "Hell-o-world"`
 - Create an alias of your command(s): `alias os1="ssh <ONID>@server-addr"`
 - ... (more)

EDITORS (VIM)

- Basic functions
 - Open a file: `vim <filename>`
 - Two modes – command and edit modes
 - Command modes
 - Store the file: `:w` / Exit: `:q` / Store and exit: `:wq`
 - Insert mode: `i` / Insert in a newline: `o`
 - Remove texts: `d <up-arrow|down-arrow>`, `dd`, `u`
 - Undo the edits: `u`
 - Search texts: `/<rexpr>`
 - Replace texts: `:%s/<old-rexpr>/<new-rexpr>/g`
 - Copy and paste texts: `yy`, `d<#lines> + <up/down>`, `p`
 - Edit modes
 - Others
 - Split screens: `:sp`, `ctrl+w+v` / Move a cursor between screens: `ctrl+w+w`

EDITORS (VIM) – CONT'D

- Let's write an example program “overflow.c”

VERSION CONTROL + EDITOR (VIM)

- A sample C program: overflow.c

```
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>

static int buffer_size = 10;

int store_name_and_print(char *buffer, char *sinput)
{
    if (sinput == NULL) {
        printf("Error: the argument string is NULL, abort.\n");
        return -1;
    }

    // copy the string to my buffer
    strcpy(buffer, sinput);

    // check what's in the buffer
    printf("My buffer holds: %s\n", buffer);

    // Here, as a CS student, we will do something with buffer...

    return 0;
}
```

```
// continue from the left...

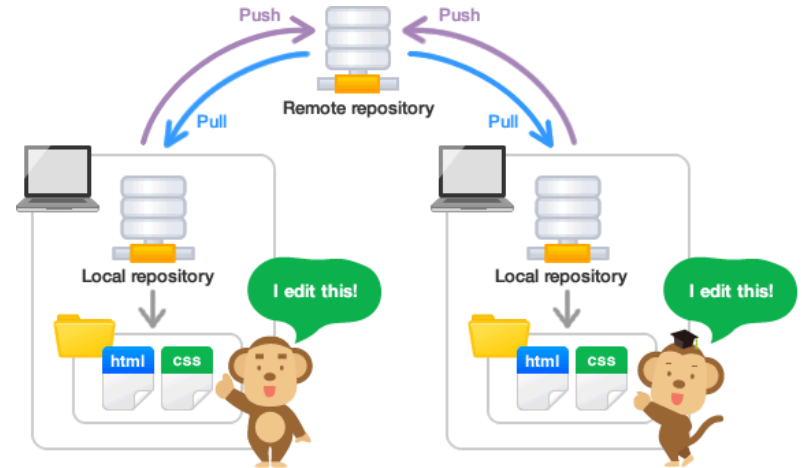
int main(int argc, char *argv[])
{
    char *buffer = (char *) malloc(buffer_size);
    int ret = 0;

    // print your name in the argument
    if (argc == 2) {
        ret = store_name_and_print(buffer, argv[1]);
    }
    else if (argc > 2) {
        printf("Error: too many arguments are given - %d, abort.\n", argc);
        return -E2BIG;
    }
    else {
        printf("Error: no name given, abort.\n");
        return -1;
    }

    return ret;
}
```

VERSION CONTROL

- How can we use?
 - Let's do an exercise with [GitHub](#)
 - Create a repository for the homework
 - Create a file and modify its content
 - Save the file and push to the repository
 - Git commands we will use:
 - `$ git clone <a remote repository url>`
 - `$ git add <files or a dir>`
 - `$ git commit -m "message"`
 - `$ git push`
 - `$ git pull`



TOPICS FOR TODAY

- Preliminaries
 - Connect to OS I server
 - Shell + script
 - Version control and editors (vim)
 - C Reviews
 - Debugging (GDB)

C REVIEW

- Revisit the sample C Program: overflow.c

```
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>
```

The function takes mem. addr.
of the arguments, not the contents
(call by reference, not call by values)

```
static int buffer_size = 10;
```

```
int store_name_and_print(char *buffer, char *sinput)
```

```
{
    if (sinput == NULL) {
        printf("Error: the argument string is NULL, abort.\n");
        return -1;
    }
```

Copy the string argument to
the buffer we allocated (10 bytes)

```
// copy the string to my buffer
```

```
strcpy(buffer, sinput);
```

```
// check what's in the buffer
```

```
printf("My buffer holds: %s\n", buffer);
```

```
// TODO: as a CS student, we will do something with buffer...
```

```
return 0;
```

```
}
```

```
// continue from the left...
```

```
int main(int argc, char *argv[])
```

```
{
    char *buffer = (char *) malloc(buffer_size);
    int ret = 0;
```

Request OS to allocate mem.
the size is equal to 10 bytes
(malloc system call)

```
// print your name in the argument
```

```
if (argc == 2) {
```

This program gets a single argument

```
    ret = store_name_and_print(buffer, argv[1]);
```

```
}
```

```
else if (argc > 2) {
```

```
    printf("Error: too many arguments are given - %d, abort.\n", argc);
```

```
    return -E2BIG;
```

If it has more than one arg., return error.

```
}
```

```
else {
```

```
    printf("Error: no name given, abort.\n");
```

```
    return -1;
```

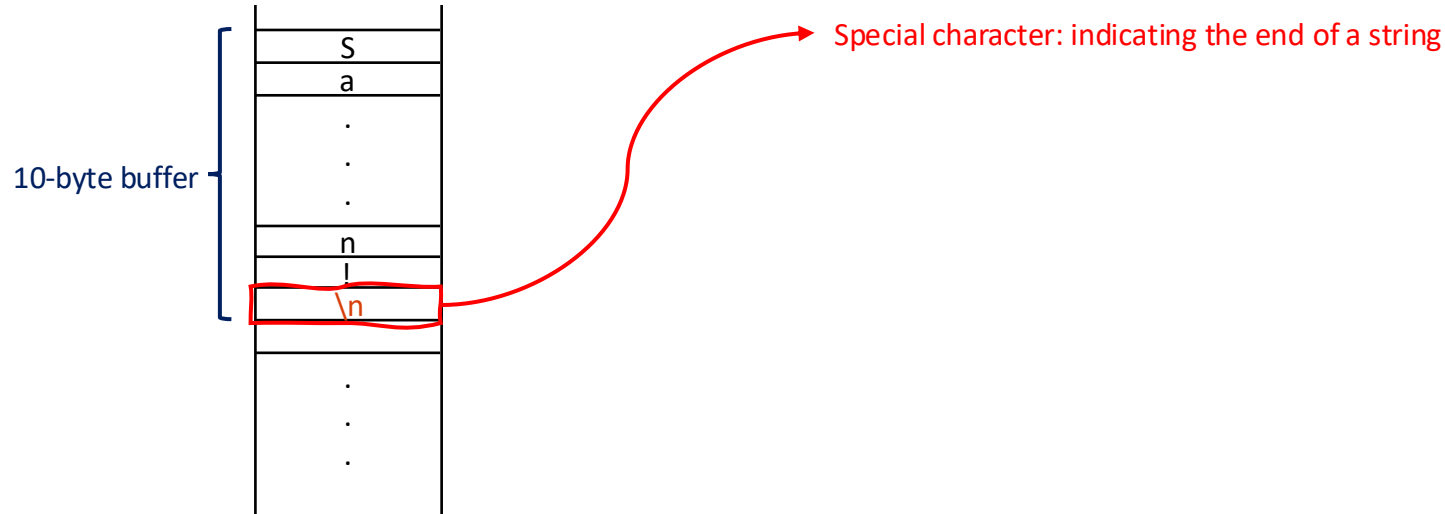
```
}
```

```
return ret;
```

```
}
```

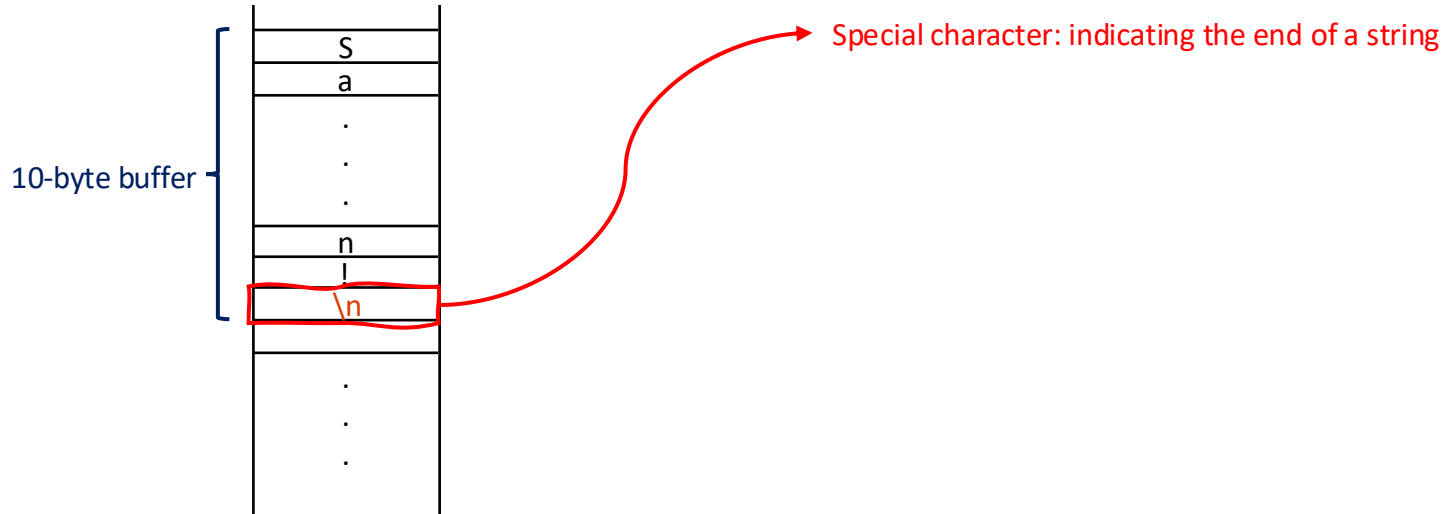
C REVIEW: HOW C STORE STRING IN MEMORY

- Sample C program: overflow.c



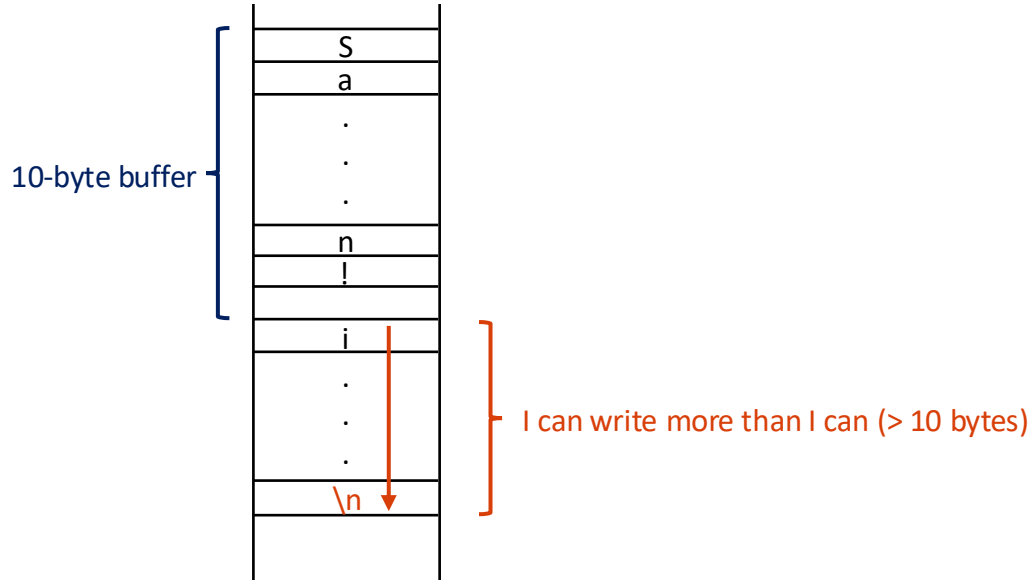
C REVIEW: C DOES **NOT** MANAGE MEMORY AUTOMATICALLY

- Sample C program: overflow.c
 - `len(sinput) < 10`: We're okay
 - `len(sinput) >= 10`: It overwrites some unknown memory locations



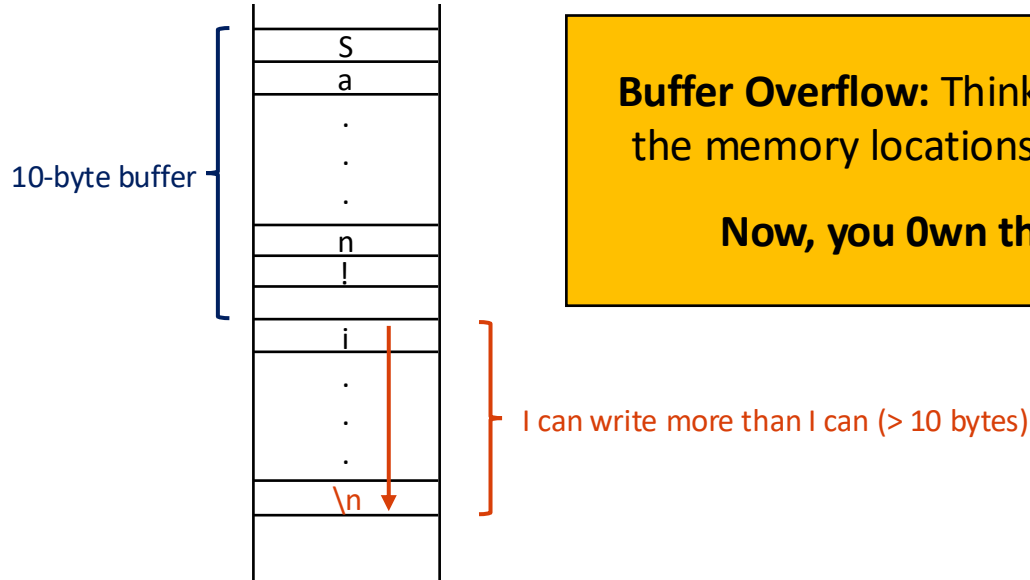
C REVIEW: BUFFER OVERFLOW SECURITY VULNERABILITY

- Sample C program: overflow.c
 - `len(sinput) < 10`: We're okay
 - `len(sinput) >= 10`: It overwrites some unknown memory locations



C REVIEW: BUFFER OVERFLOW SECURITY VULNERABILITY

- Sample C program: overflow.c
 - `len(sinput) < 10`: We're okay
 - `len(sinput) >= 10`: It overwrites some unknown memory locations



Buffer Overflow: Think about you wrote a program [?!] at the memory locations > 10 bytes, and that's executable!

Now, you Own the system (Security Problem)!

C REVIEW: SOLUTION

- Secure programming practices

```
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>

static int buffer_size = 10;

int store_name_and_print(char *buffer, char *sinput)
{
    if (sinput == NULL) {
        printf("Error: the argument string is NULL, abort.\n");
        return -1;
    }

    // copy the string to my buffer
    strncpy(buffer, sinput, buffer_size);

    // check what's in the buffer
    printf("My buffer holds: %s\n", buffer);

    // TODO: as a CS student, we will do something with buffer...

    return 0;
}
```

Copy the string exactly 10 bytes
and then truncate the rest of it!

// continue from the left...

```
int main(int argc, char *argv[])
{
    char *buffer = (char *) malloc(buffer_size);
    int ret = 0;

    // print your name in the argument
    if (argc == 2) {
        ret = store_name_and_print(buffer, argv[1]);
    }
    else if (argc > 2) {
        printf("Error: too many arguments are given - %d, abort.\n", argc);
        return -E2BIG;
    }
    else {
        printf("Error: no name given, abort.\n");
        return -1;
    }

    return ret;
}
```

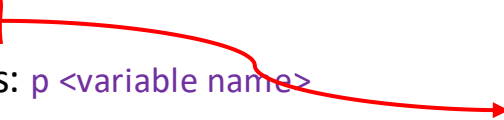
TOPICS FOR TODAY

- Preliminaries
 - Connect to OS I server
 - Shell + script
 - Version control and editors (vim)
 - C Reviews
 - Debugging (GDB)

DEBUGGING WITH GDB

- Types of errors we will face:
 - Static errors, such as syntax errors
 - Relatively easy to fix; GCC provides error messages
 - Runtime errors, such as buffer overflow:
 - Hard to fix
 - Program runs, but does not provide the expected outputs
 - ...

DEBUGGING WITH GDB

- [GDB](#): a tool for debugging C programs in runtime
 - **Pre-requisite:**
 - Compile our program with debug symbols (-g): `gcc -g <source file> -o <output file>`
 - Run the executable with gdb: `gdb ./<output file>`
 - **Useful commands:**
 - See lines of codes: `list <line #>`, `list`
 - Breakpoints: `break <line #>`
 - Run: `run` / `step` (if you want to execute one line of code at a time)
 - Backtrace: `bt` 
 - Print variables: `p <variable name>`
 - Clear the screen: `ctrl + l`
 - ... [More](#)
- This bt command prints out a list of functions called
The list of fn will be printed as FILO order like “stack”
#0 store_name_and_print
#1 main

BUFFER OVERFLOW EXPLOIT

- Sample exploit: subvert.c

```
(base) os1 ~/temp 1012$ ./subvert  
Enter your password:  
thesecretbuff  
Correct password, login!  
Now you are allowed to run some private queries  
(base) os1 ~/temp 1013$ ./subvert  
Enter your password:  
hackerishere  
[Error] incorrect password
```

```
(base) os1 ~/temp 1013$ ./subvert  
Enter your password:  
hhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhhh  
[Error] incorrect password  
Now you are allowed to run some private queries  
Segmentation fault
```

```
(base) os1 ~/temp 1013$ █
```

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char buf[15];
    int pass = 0;

    // read a name from the command line
    printf("Enter your password: \n");
    gets(buf);

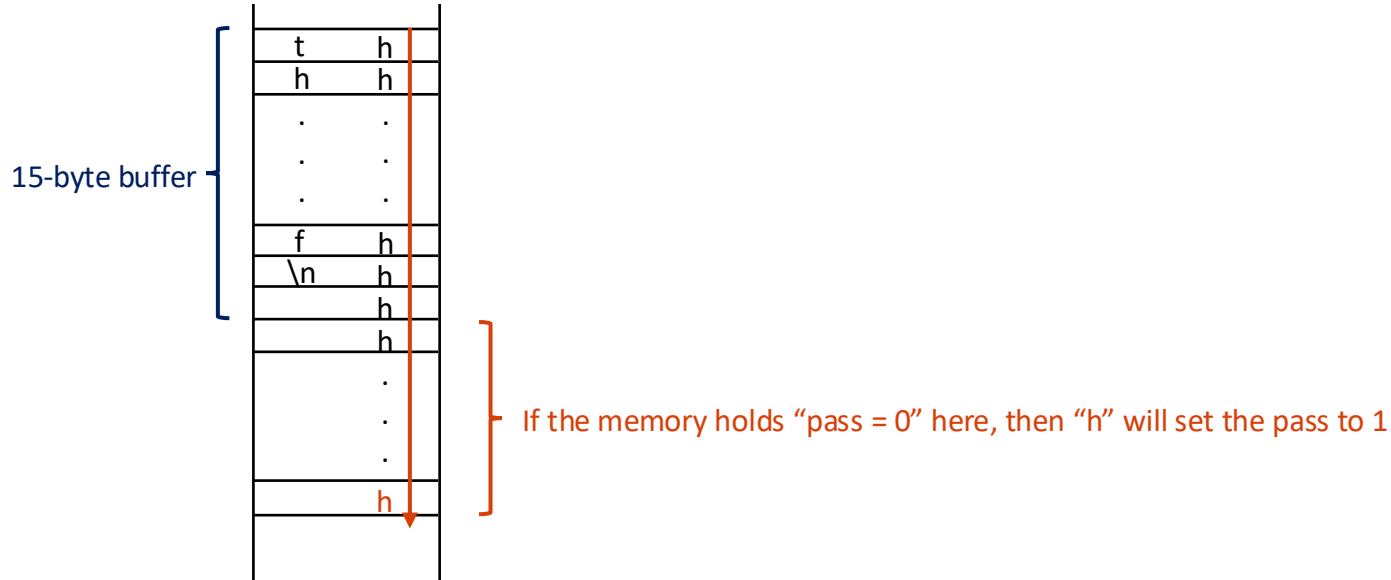
    if (strcmp(buf, "thesecretbuff")) {
        printf("[Error] incorrect password\n");
    }
    else {
        printf("Correct password, login!\n");
        pass = 1;
    }

    // read a ssn from the command line
    if (pass) {
        printf("Now you are allowed to run some private queries\n");
    }

    return 0;
}
```


BUFFER OVERFLOW EXPLOIT

- Sample C program: subvert.c
 - **Normal:** the password I type will be stored into the 15-byte buffer
 - **Attack:** the password “hhhhh...hhh” will go over the 15-byte limit
 - **Real-world cases:** [Heartbleed](#), [Shellshock](#)



DEBUG THIS CODE WITH GDB

- Let's inspect the buffer status with GDB

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char buf[15];
    int pass = 0;

    // read a name from the command line
    printf("Enter your password: \n");
    gets(buf);

    if (strcmp(buf, "thesecretbuff")) {
        printf("[Error] incorrect password\n");
    }
    else {
        printf("Correct password, login!\n");
        pass = 1;
    }

    // read a ssn from the command line
    if (pass) {
        printf("Now you are allowed to run some private queries\n");
    }

    return 0;
}
```

TOPICS FOR TODAY

- Preliminaries
 - Connect to OS I server
 - Shell + script
 - Version control and editors (vim)
 - C Reviews
 - Debugging (GDB)

VERSION CONTROL

VERSION CONTROL

- Problems we may face
 - What if we do accidentally `rm -rf <project dir>`?
 - What if our computer **suddenly not working**?
 - What if we **remove a piece of code** that was correct?
- Consequences
 - Give up
 - Re-write the code from scratch

VERSION CONTROL

- Solution: use version control tools
 - **Definition:** the practice of tracking and managing changes to source code
 - **Available tools:**
 - Github
 - Gitlab
 - Git
 - Bitbucket
 - Microsoft Team Foundation
 - ...

Top Source Code Management Technologies

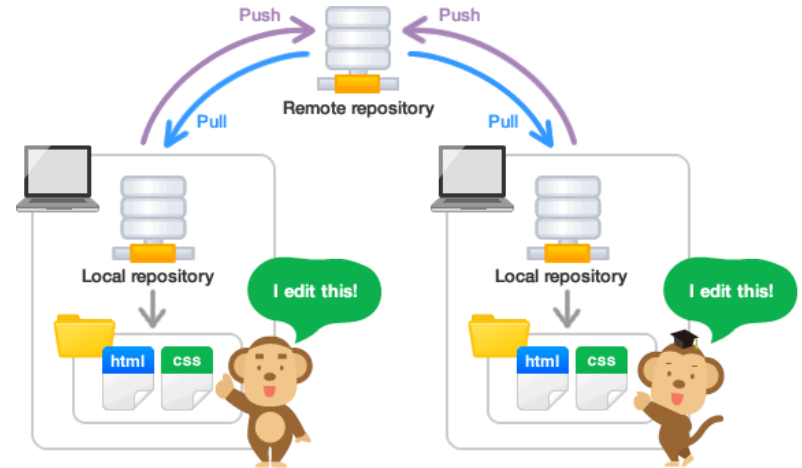
Github has the top spot for best Source Code Management by market share. Followed by Git and Bitbucket

- 1  Github
- 2  Git
- 3  Bitbucket
- 4  Microsoft Team Foundation Server (TFS)
- 5  GitLab



VERSION CONTROL

- How can we use?
 - Let's do an exercise with [GitHub](#)
 - Create a repository for the homework
 - Create a file and modify its content
 - Save the file and push to the repository
 - Git commands we will use:
 - `$ git clone <a remote repository url>`
 - `$ git add <files or a dir>`
 - `$ git commit -m "message"`
 - `$ git push`
 - `$ git pull`



VERSION CONTROL – GIT WITH SSH

- SSH without password
 - Authentication using an SSH key
 - **Pro:** don't need to type password in every SSH log-in
- How to?
 - Generate a *private* and *public* key pair on your PC/laptop
 - Command: `ssh-keygen -t ed25519 -C "<ONID>@oregonstate.edu"`
 - Output: you will have <keyname> and <keyname>.pub under a specified folder
 - Copy the public key to the OS 1 server
 - Open <keyname>.pub and copy the content
 - Paste it into *authorized_keys* file in OS 1 server's <your home>/*.ssh* folder
 - Update the permission of *authorized_keys* file to 700: `chmod 0700 authorized_key`
 - Try SSH command again
 - `ssh <ONID>@access/flip.engr.oregonstate.edu` (It won't ask the password again)

VERSION CONTROL

- Some more tips
 - `.gitignore`
 - Merge conflict
 - Git pull before add, commit and push
 - Advanced CI (not needed for OS1 homework assignments)
 - Create a branch
 - Modify and commit your code
 - Make a pull request

Thank You!

Mon/Wed 12:00 – 1:50 PM (LINC #2000)

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI